

Refactoring Techniques and Strategies (a Tale in Three Acts)

DAVID STEFFEN

Dave Steffen

Principal Software Engineer

dsteffen@scitec.com



Act 0: The Basics

Act 1: The Problem

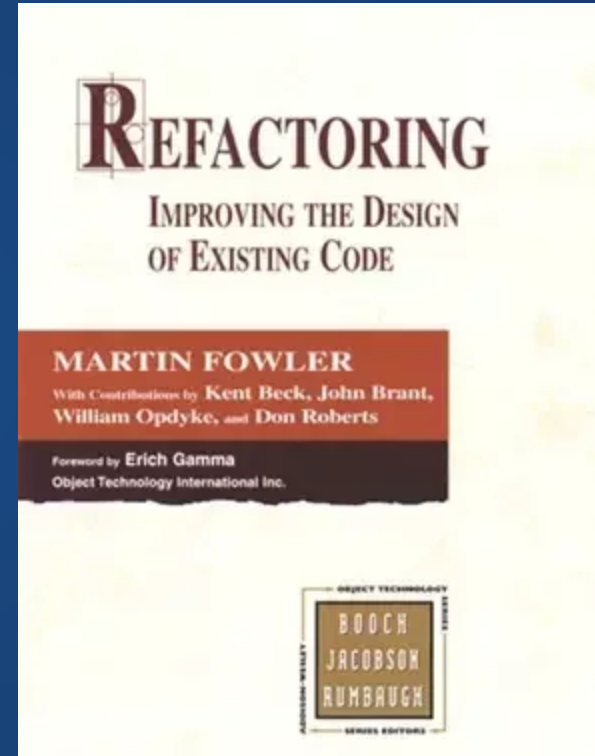
Act 2: The Work

Act 3: The Sell

Refactoring

Refactoring (noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior

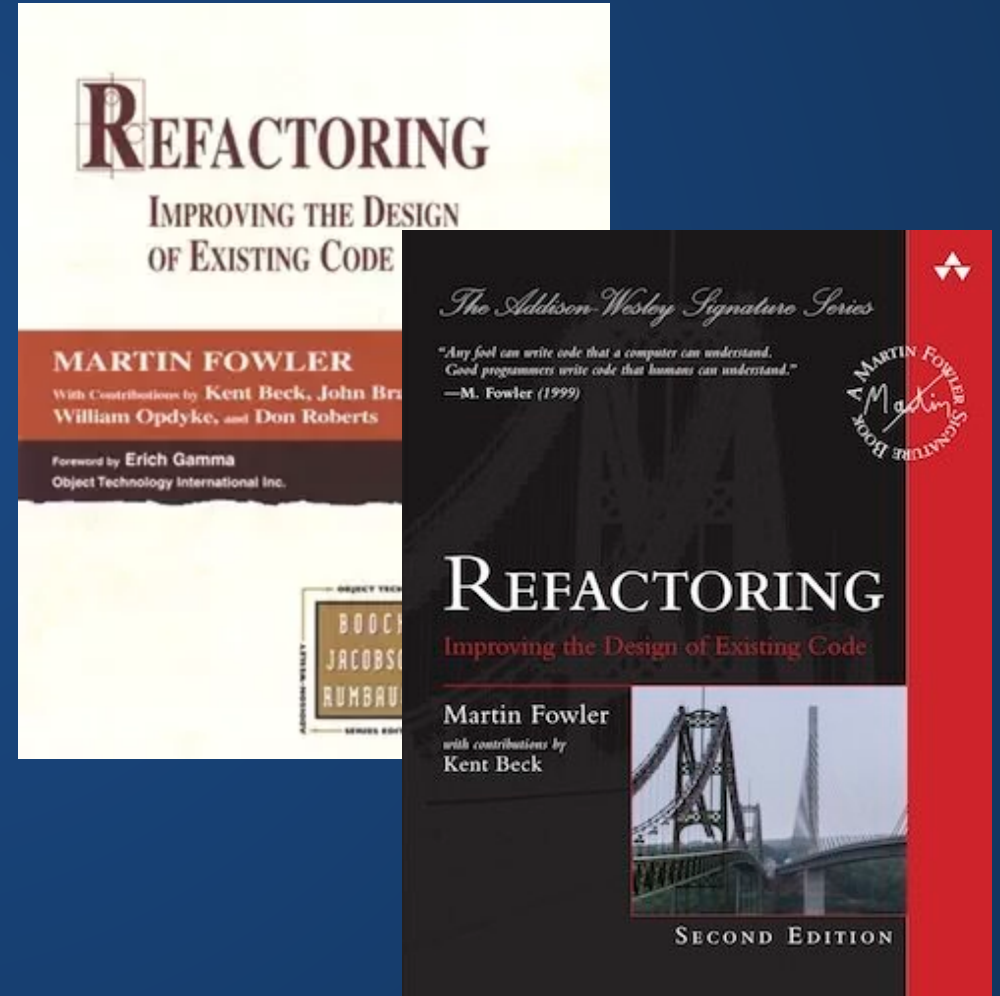
-- Martin Fowler 1999



Refactoring

Refactoring (noun): a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior

-- Martin Fowler 1999
2018



Fowler's Catalogs

<https://refactoring.com/catalog/>

Code Smells

Fowler and Kent Beck also presented a list of "Code Smells":

Code Smells

Fowler and Kent Beck also presented a list of "Code Smells":

"If it stinks, change it"

Grandma Beck, discussing child-rearing philosophy

Code Smells

Fowler and Kent Beck also presented a list of "Code Smells":

"If it stinks, change it"

Grandma Beck, discussing child-rearing philosophy

These are hints or clues that something is wrong.

Code Smells are not bugs

They're not necessarily what's wrong

They're just hints that something *might* be amiss.

Code Smells

(2nd Ed version)

- Mysterious Name
- Duplicated Code
- Long Function
- Long Parameter List
- Global Data
- Mutable Data
- Divergent Change ("Churn Pot")
- Shotgun Surgery
- Feature Envy
- Data Clumps
- Primitive Obsession
- Repeated Switches
- Loops
- Lazy Element
- Speculative Generality
- Temporary Field
- Message Chains
- Middle Man
- Insider Trading
- Large Class
- Alternative Classes with Different Interfaces
- Data Class
- Refused Bequest
- Comments

Code Smells

(2nd Ed version)

- **Mysterious Name**
- **Duplicated Code**
- **Long Function**
- Long Parameter List
- Global Data
- Mutable Data
- Divergent Change ("Churn Pot")
- Shotgun Surgery
- Feature Envy
- **Data Clumps**
- Primitive Obsession
- Repeated Switches
- **Loops**
- Lazy Element
- Speculative Generality
- Temporary Field
- Message Chains
- Middle Man
- Insider Trading
- Large Class
- Alternative Classes with Different Interfaces
- Data Class
- Refused Bequest
- **Comments**

Code Smells

(2nd Ed version)

- **Mysterious Name**
- **Duplicated Code**
- **Long Function**
- Long Parameter List
- Global Data
- Mutable Data
- Divergent Change ("Churn Pot")
- Shotgun Surgery
- Feature Envy
- **Data Clumps**
- Primitive Obsession
- Repeated Switches
- **Loops**
- Lazy Element
- Speculative Generality
- Temporary Field
- Message Chains
- Middle Man
- Insider Trading
- Large Class
- Alternative Classes with Different Interfaces
- Data Class
- Refused Bequest
- **Comments**
 - **Code Smell? or Perfume?**

Refactoring Goals

Improve the architecture, design,
and structure of the code.

- Not bugfixes
- Not optimizations
- Increased Maintainability:
 - easier to understand
 - cheaper to modify

Underlying Themes

- Simplification
 - Single Responsibility Rule
 - Increase Abstraction
-
- Abstraction: "The cognitive process of isolating common features or essential qualities from specific, concrete objects or experiences while ignoring irrelevant details"

Underlying Themes

- Simplification
 - Single Responsibility Rule
 - Increase Abstraction
- Abstraction: "The cognitive process of isolating common features or essential qualities from specific, concrete objects or experiences while ignoring irrelevant details"

Kate Gregory, "Keynote: Abstraction Patterns" NDC TechTown 2022

<https://youtu.be/rfIX0FzKHF0>

Underlying Themes

- Simplification
 - Single Responsibility Rule
 - Increase Abstraction
- Abstraction: "The cognitive process of isolating common features or essential qualities from specific, concrete objects or experiences while ignoring irrelevant details"

Kate Gregory, "Keynote: Abstraction Patterns" NDC TechTown 2022

<https://youtu.be/rfIX0FzKHF0>

- Abstraction:
 - Something with a name
 - Something with a specific function or meaning
 - Reduces repetition and duplication

Refactoring:

Rename Variable/method/class

Refactoring:

Rename Variable/method/class

Simplest, most obvious; but arguably the most powerful, for non-obvious reasons

Kate Gregory, "Naming is Hard: Let's Do Better" CppCon 2019

<https://youtu.be/MBRoCdtZOYg>

Required viewing

"Mysterious Name" code smell

```
double d1, d2, d3;
```

"Mysterious Name" code smell

```
double d1, d2, d3;
```

- Readability
 - Giving things good names increases the information content of the code

"Mysterious Name" code smell

```
double d1, d2, d3;
```

- Readability
 - Giving things good names increases the information content of the code
- Single letter variables are *fine* if everyone knows what they mean

```
int i,j,k;           // i,j,k as indices are as old as the hills  
auto F = m * a;      // Newton's laws are 339 years old
```

"Mysterious Name" code smell

```
double d1, d2, d3;
```

- Readability
 - Giving things good names increases the information content of the code
- Single letter variables are *fine* if everyone knows what they mean

```
int i,j,k;           // i,j,k as indices are as old as the hills  
auto F = m * a;      // Newton's laws are 339 years old
```

Use the terminology of the domain you're in.
It's the common language of the people you're working with.

Goals

- Maintainability:
 - easier to understand
 - cheaper to modify *correctly*
- *Increase Readability*
 - *Increase the information content*

Rename <thing>

Names have Power

Rename <thing>

Names have Power

Kate Gregory, Emotional Code ACCU 2019 1:00:24

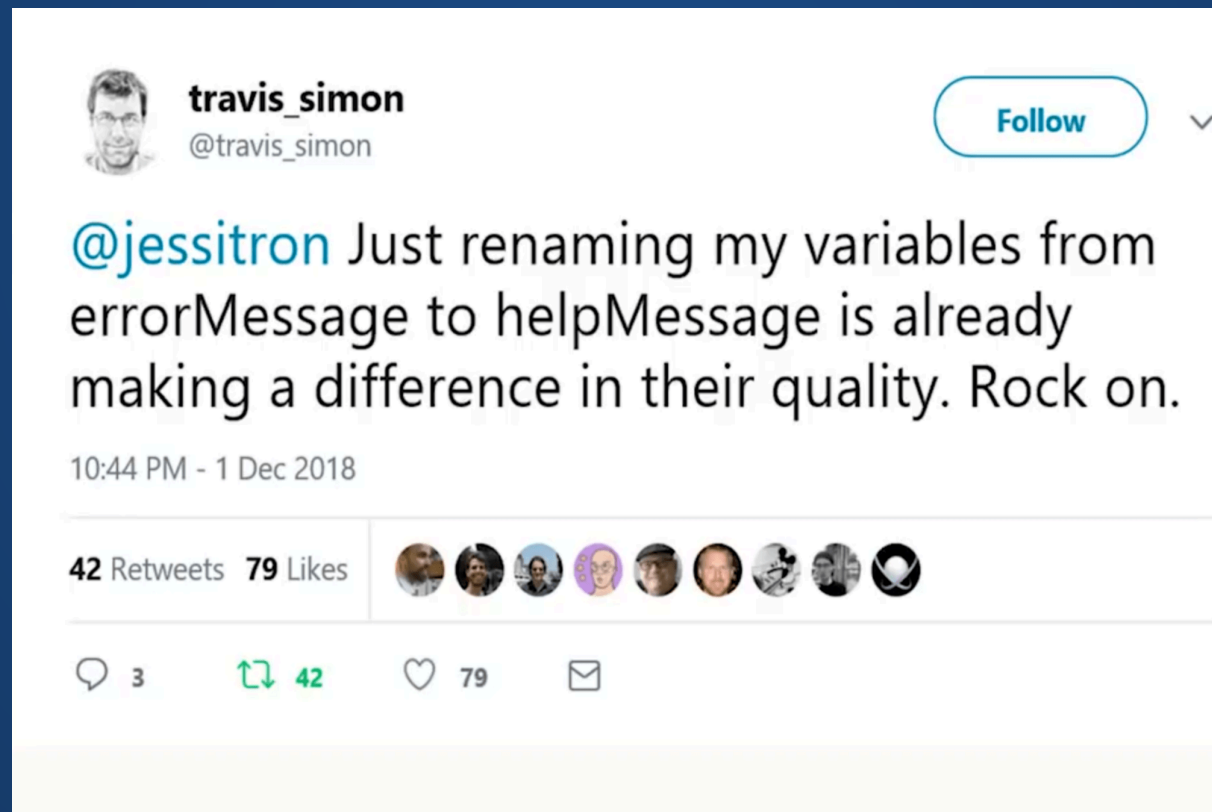
<https://youtu.be/uloVXmSHiSo?t=3642>

Rename <thing>

Names have Power

Kate Gregory, Emotional Code ACCU 2019 1:00:24

<https://youtu.be/uloVXmSHiSo?t=3642>



To Rename is To Reframe

- "Error message" → "help message"

To Rename is To Reframe

- "Error message" → "help message"
- "script" → "program"

To Rename is To Reframe

- "Error message" → "help message"
- "script" → "program"
- "bug" → "error" : (Dijkstra)

Edsger W. Dijkstra's request to programmers <https://youtu.be/606Vxh-jQZY>

Naming Wrapup

- Good names are *hard*
 - Ask your friendly neighborhood LLM for help?
- Good names increase readability
- Good names increase information content
- Good names help future developers (including you)
- Good names change the frame

Refactoring:

Extract Function / Method

Take some code and extract it into a separate function
with a name

- Duplicated code (undo cut-n-paste code)
 - or *nearly* duplicated code

Duplicate Code : "Log and Throw"

Kevlin Henney "Clean Coders Hate What Happens to Your Code When You Use These Enterprise Programming Tricks" NDC Conferences 2017

<https://youtu.be/FyCYva9Dhsl>

Duplicate Code : "Log and Throw"

Kevlin Henney "Clean Coders Hate What Happens to Your Code When You Use These Enterprise Programming Tricks" NDC Conferences 2017

<https://youtu.be/FyCYva9Dhsl>

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK (throw if it's not)
4     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
5     if (cfgAddress.empty()) {
6         auto msg = std::string{"Configuration Value Missing : address"};
7         Log::Instance() << msg();
8         throw ConnectionException(msg);
9     }
10 }
```

Duplicate Code : "Log and Throw"

Kevlin Henney "Clean Coders Hate What Happens to Your Code When You Use These Enterprise Programming Tricks" NDC Conferences 2017

<https://youtu.be/FyCYva9Dhsl>

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK (throw if it's not)
4     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
5     if (cfgAddress.empty()) {
6         auto msg = std::string{"Configuration Value Missing : address"};
7         Log::Instance() << msg();
8         throw ConnectionException(msg);
9     }
10 }
```

Duplicate Code : "Log and Throw"

```
1 Connection* CreateServerConnection() {
2     std::ostringstream buf;
3
4     // Step 1: Get address and check that it's OK (throw if it's not)
5     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
6     if (cfgAddress.empty()) {
7         buf << std::format("Configuration Value Missing {}", "address");
8         Log::Instance() << buf.str();
9         throw ConnectionException(buf.str());
10    }
11
12    // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
13    auto address = inet_addr(cfgAddress.c_str());
14    if (address == -1) {
15        buf << std::format("Invalid Address: {}", cfg_address);
16        Log::Instance() << buf.str();
17        throw ConnectionException(buf.str());
18    }
19
20    // Step 3: Get port and check that its OK (throw if it's not)
21    std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
22    if (cfgPort.empty()) {
23        buf << std::format("Configuration value missing : {}", "port");
24        Log::Instance() << buf.str();
25        throw ConnectionException(buf.str());
26    }
```

Duplicate Code : "Log and Throw"

```
3
4 // Step 1: Get address and check that it's OK (throw if it's not)
5 std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
6 if (cfgAddress.empty()) {
7     buf << std::format("Configuration Value Missing {}", "address");
8     Log::Instance() << buf.str();
9     throw ConnectionException(buf.str());
10 }
11
12 // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
13 auto address = inet_addr(cfgAddress.c_str());
14 if (address == -1) {
15     buf << std::format("Invalid Address: {}", cfg_address);
16     Log::Instance() << buf.str();
17     throw ConnectionException(buf.str());
18 }
19
20 // Step 3: Get port and check that its OK (throw if it's not)
21 std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
22 if (cfgPort.empty()) {
23     buf << std::format("Configuration value missing : {}", "port");
24     Log::Instance() << buf.str();
25     throw ConnectionException(buf.str());
26 }
27
28 // Step 4: Convert port to bytes
29 auto port = htons(atoi(cfgPort.c_str()));
```

Duplicate Code : "Log and Throw"

```
1 Connection* CreateServerConnection() {
2     std::ostringstream buf;
3
4     // Step 1: Get address and check that it's OK (throw if it's not)
5     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
6     if (cfgAddress.empty()) {
7         buf << std::format("Configuration Value Missing {}", "address");
8         Log::Instance() << buf.str();
9         throw ConnectionException(buf.str());
10    }
11
12    // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
13    auto address = inet_addr(cfgAddress.c_str());
14    if (address == -1) {
15        buf << std::format("Invalid Address: {}", cfg_address);
16        Log::Instance() << buf.str();
17        throw ConnectionException(buf.str());
18    }
19
20    // Step 3: Get port and check that its OK (throw if it's not)
21    std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
22    if (cfgPort.empty()) {
23        buf << std::format("Configuration value missing : {}", "port");
24        Log::Instance() << buf.str();
25        throw ConnectionException(buf.str());
26    }
```

Duplicate Code : "Log and Throw"

```
1 Connection* CreateServerConnection() {
2     std::ostringstream buf;
3
4     // Step 1: Get address and check that it's OK (throw if it's not)
5     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
6     if (cfgAddress.empty()) {
7         buf << std::format("Configuration Value Missing {}", "address");
8         Log::Instance() << buf.str();
9         throw ConnectionException(buf.str());
10    }
11
12    // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
13    auto address = inet_addr(cfgAddress.c_str());
14    if (address == -1) {
15        buf << std::format("Invalid Address: {}", cfg_address);
16        Log::Instance() << buf.str();
17        throw ConnectionException(buf.str());
18    }
19
20    // Step 3: Get port and check
21    std::string cfgPort = Configuration
22    if (cfgPort.empty()) {
23        buf << std::format("Config
24        Log::Instance() << buf.str
25        throw ConnectionException(buf.str()),
26    }
```

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs)
{
    auto msg = first + (... + strs);
    Log << msg;
    throw ConnectionException{msg};
}
```

Duplicate Code : "Log and Throw"

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... str) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // step 1: Get address and check that it's OK (throw if it's not)
4     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
5     if (cfgAddress.empty()) {
6         ReportConnectionFailure("Configuration Value Missing : address");
7
8
9     }
10
11     // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
12     auto address = inet_addr(cfgAddress.c_str());
13     if (address == -1) {
14         ReportConnectionFailure("Invalid Address : ", cfg_address);
15
16
17     }
18
19     // Step 3: Get port and check that its OK (throw if it's not)
20     std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
21     if (cfgPort.empty()) {}
22     ReportConnectionFailure("Configuration value missing : port");
23
24
25 }
26
```

Duplicate Code : "Log and Throw"

```
template<class... Strs>
```

```
auto ReportConnectionFailure(std::string const& first, Strs... str) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
5     if (cfgAddress.empty()) {
6         ReportConnectionFailure("Configuration Value Missing : address");
7
8
9     }
10
11    // Step 2: Convert address to bytes and check that it's OK
12    auto address = inet_addr(cfgAddress.c_str());
13    if (address == -1) {
14        ReportConnectionFailure("Invalid Address : ", cfg_address);
15
16
17    }
18
19    // Step 3: Get port and check that its OK
20    std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
21    if (cfgPort.empty()) {}
22        ReportConnectionFailure("Configuration value missing : port");
23
24
25    }
26
```

Duplicate Code · SafeConfiguration

```
template<class... Strs>
```

```
auto ReportConnectionFailure(std::string const& first, Strs... str) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
5     if (cfgAddress.empty()) {
6         ReportConnectionFailure("Configuration Value Missing : address");
7
8     }
9
10
11     // Step 2: Convert address to bytes and check that it's OK
12     auto address = inet_addr(cfgAddress.c_str());
13     if (address == -1) {
14         ReportConnectionFailure("Invalid Address : ", cfg_address);
15
16     }
17
18
19     // Step 3: Get port and check that its OK
20     std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
21     if (cfgPort.empty()) {}
22     ReportConnectionFailure("Configuration value missing : port");
23
24
25 }
26
```

Duplicate Code : SafeConfiguration

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
5     if (cfgAddress.empty()) {
6         ReportConnectionFailure("Configuration Value Missing : address");
7
8     }
9
10
11     // Step 2: Convert address to integer
12     auto address = inet_aton(cfgAddress.c_str(), &addr);
13     if (address == -1) {
14         ReportConnectionFailure("Configuration Value Missing : address");
15
16     }
17
18
19     // Step 3: Get port and check that it's OK
20     std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
21     if (cfgPort.empty()) {}
22     ReportConnectionFailure("Configuration value missing : port");
23
24
25 }
26
```

```
auto GetSafeConfiguration(std::string_view val) -> std::string {
    auto ret = ConfigurationManager::Instance().GetValue(val);
    if (ret.empty())
        ReportConnectionFailure("Configuration Value Missing: " + val);
    return ret;
}
```

Duplicate Code · SafeConfiguration

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... str) {...}
```

```
1 Connection* CreateServerConnection
2
3 // Step 1: Get address and check
4 std::string cfgAddress = GetSafeConfiguration("address");
5
6
7
8
9
10
11 // Step 2: Convert address to bytes and check that it's OK
12 auto address = inet_addr(cfgAddress.c_str());
13 if (address == -1) {
14     ReportConnectionFailure("Invalid Address : ",cfg_address);
15
16
17 }
18
19 // Step 3: Get port and check that its OK
20 std::string cfgPort = GetSafeConfiguration("port");
21
22
23
24
25
26
```

Duplicate Code · SafeConfiguration

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... str) {...}
```

```
auto GetConfiguration(std::string_view val) -> std::string;
```

```
1 Connection* CreateServerConnection
2
3 // Step 1: Get address and check
4 std::string cfgAddress = GetConfiguration("address");
5
6
7
8
9
10
11 // Step 2: Convert address to bytes and check that it's OK
12 auto address = inet_addr(cfgAddress.c_str());
13 if (address == -1) {
14     ReportConnectionFailure("Invalid Address : ", cfg_address);
15
16
17 }
18
19 // Step 3: Get port and check that it's OK
20 std::string cfgPort = GetConfiguration("port");
21
22
23
24
25
26
```

Duplicate Code · SafeConfiguration

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = inet_addr(cfgAddress.c_str());
8     if (address == -1)
9         ReportConnectionFailure("Invalid Address : ", cfg_address);
10
11    // Step 3: Get port and check that its OK
12    std::string cfgPort = GetConfiguration("port");
13
14    // Step 4: convert port to bytes
15    auto port = htons(atoi(cfgPort.c_str()));
16
17    // Step p5: Create connection and check that it's OK
18    auto result = new connection(address.port);
19    if (!result || !result->IsOK())
20        ReportConnectionFailure("Failed to connect : ", cfgAddres, ':', cfgPort);
21
22    return result;
23 }
```

Goals

- Maintainability:
 - easier to understand
 - cheaper to modify *correctly*
- Increase Readability
 - Increase the information content
- *Identify Abstractions*
 - *Simplify*
 - *Decrease amount of code*

Long Function

What is "Long"?

- 5000 lines

Long Function

What is "Long"?

- 5000 lines
- 1000 lines

Long Function

What is "Long"?

- 5000 lines
- 1000 lines
- 100 lines

Long Function

What is "Long"?

- 5000 lines
- 1000 lines
- 100 lines
- 50

Long Function

What is "Long"?

- 5000 lines
- 1000 lines
- 100 lines
- 50

and what if there's no duplicated code?

It's not repetitive, it's just big!

Long Function

Long Function

Tony Van Eerd, "Postmodern C++" CppCon 2017

- <https://youtu.be/QTln3goa3A8>

Long Function

Tony Van Eerd, "Postmodern C++" CppCon 2017

- <https://youtu.be/QLn3goa3A8>

How many times must the same code be seen
Before you make it a subroutine?

Long Function

Tony Van Eerd, "Postmodern C++" CppCon 2017

- <https://youtu.be/QTln3goa3A8>

How many times must the same code be seen
Before you make it a subroutine?

Answer

1

Long Function

Tony Van Eerd, "Postmodern C++" CppCon 2017

- <https://youtu.be/QTln3goa3A8>

How many times must the same code be seen
Before you make it a subroutine?

Answer

1

You don't need it to be duplicated at all
in order to elevate it to a function call

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

```

```

1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = inet_addr(cfgAddress.c_str());
8     if (address == -1)
9         ReportConnectionFailure("Invalid Address : ", cfg_address);
10
11    // Step 3: Get port and check that its OK
12    std::string cfgPort = GetConfiguration("port");
13
14    // Step 4: Convert port to bytes
15    auto port = htons(atoi(cfgPort.c_str()));
16
17    // Step 5: Create connection and check that it's OK
18    auto result = new connection(address, port);
19    if (!result || !result->IsOK())
20        ReportConnectionFailure("Failed to connect : ", cfgAddress, ':', cfgPort);
21
22    return result;
23 }

```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

```

```

1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = inet_addr(cfgAddress.c_str());
8     if (address == -1)
9         ReportConnectionFailure("Invalid Address : ", cfg_address);
10
11    // Step 3: Get port and check that its OK
12    std::string cfgPort = GetConfiguration("port");
13
14    // Step 4: Convert port to byte
15    auto port = htons(atoi(cfgPort.c_str()));
16
17    // Step 5: Create connection at
18    auto result = new connection(address, port);
19    if (!result || !result->IsOK())
20        ReportConnectionFailure("Failed to create connection", "address", "port");
21
22    return result;
23 }

```

```

auto CheckedInetAddr (std::string const& cfgAddress) {
    auto address = inet_addr(cfgAddress.c_str());
    if (address == -1) {
        ReportConnectionFailure("Invalid Address: ", cfg_address);
    }
    return address;
}

```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}

```

```

1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = CheckedInetAddr(cfgAddress)
8
9     // Step 3: Get port and check that its OK
10    std::string cfgPort = GetConfiguration("port");
11
12    // Step 4: Convert port to bytes
13    auto port = htons(atoi(cfgPort.c_str()));
14
15    // Step 5: Create connection and check that it's OK
16    auto result = new connection(address, port);
17    if (!result || !result->IsOK())
18        ReportConnectionFailure("Failed to connect : ", cfgAddress, ':', cfgPort);
19
20    return result;
21 }

```

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = CheckedInetAddr(cfgAddress)
8
9     // Step 3: Get port and check that its OK
10    std::string cfgPort = GetConfiguration("port");
11
12    // Step 4: Convert port to bytes
13    auto port = htons(atoi(cfgPort.c_str()));
14
15    // Step 5: Create connection and check that it's OK
16    auto result = new connection(address, port);
17    if (!result || !result->IsOK())
18        ReportConnectionFailure("Failed to connect : ", cfgAddress, ':', cfgPort);
19
20    return result;
21 }
```

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = CheckedInetAddr(cfgAddress)
8
9     // Step 3: Get port and check that its OK
10    std::string cfgPort = GetConfiguration("port");
11
12    // Step 4: Convert port to bytes
13    auto port = htons(atoi(cfgPort.c_str()));
14
15    // Step 5: Create connection and check that it's OK
16    auto result = new connection(address, port);
17    if (not result->IsOK())
18        ReportConnectionFailure("Failed to connect : ", cfgAddress, ':', cfgPort);
19
20    return result;
21 }
```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}

```

```

1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = CheckedInetAddr(cfgAddress)
8
9     // Step 3: Get port and check that its OK
10    std::string cfgPort = GetConfiguration("port");
11
12    // Step 4: Convert port to bytes
13    auto port = htons(atoi(cfgPort.c_str()));
14
15    // Step 5: Create connection and check that it's OK
16    auto result = new connection(address, port);
17    if (not result->IsOK())
18        ReportConnectionFailure("Failed to connect : ", cfgAddress, ':', cfgPort);
19
20    return result;
21 }

```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}

```

```

1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes
7     auto address = CheckedInetAddr(cfgAd
8
9     // Step 3: Get port and check that i
10    std::string cfgPort = GetConfigurati
11
12    // Step 4: Convert port to bytes
13    auto port = htons(atoi(cfgPort.c_str
14
15    // Step 5: Create connection and check that it's OK
16    auto result = new connection(address, port);
17    if (not result->IsOK())
18        ReportConnectionFailure("Failed to connect : ", cfgAddres, ':', cfgPort);
19
20    return result;
21 }

```

```

auto MakeConnection(auto address, auto port) {
    auto result = new connection(address.port);
    if (not result->IsOK())
        ReportConnectionFailure("Failed to connect : ",
                                cfgAddress, ':', cfgPort);
    return result;
}

```

```
template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}
```

```
auto GetConfiguration(std::string_view val) -> std::string;
```

```
auto CheckedInetAddr (std::string const& cfgAddress) {...}
```

```
1 Connection* CreateServerConnection() {
2
3     // Step 1: Get address and check that it's OK
4     std::string cfgAddress = GetConfiguration("address");
5
6     // Step 2: Convert address to bytes and check that it's OK
7     auto address = CheckedInetAddr(cfgAddress)
8
9     // Step 3: Get port and check that its OK
10    std::string cfgPort = GetConfiguration("port");
11
12    // Step 4: Convert port to bytes
13    auto port = htons(atoi(cfgPort.c_str()));
14
15    // Step 5: Create connection and check that it's OK
16    auto result = MakeConnection(address, port);
17
18    return result;
19 }
```

```
auto MakeConnection(auto address, auto port) {...}
```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}

auto MakeConnection(auto address, auto port) {...}

```

```

1 Connection* CreateServerConnection() {
2
3     std::string cfgAddress = GetConfiguration("address");
4
5     auto address = CheckedInetAddr(cfgAddress)
6
7     std::string cfgPort = GetConfiguration("port");
8
9     auto port = htons(atoi(cfgPort.c_str()));
10
11     return MakeConnection(address, port);
12 }

```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}

auto MakeConnection(auto address, auto port) {...}

```

```

1 auto CreateServerConnection() {
2
3     std::string cfgAddress = GetConfiguration("address");
4
5     auto address = CheckedInetAddr(cfgAddress)
6
7     std::string cfgPort = GetConfiguration("port");
8
9     auto port = htons(atoi(cfgPort.c_str()));
10
11     return MakeConnection(address, port);
12 }

```

```

template<class... Strs>
auto ReportConnectionFailure(std::string const& first, Strs... strs) {...}

auto GetConfiguration(std::string_view val) -> std::string;

auto CheckedInetAddr (std::string const& cfgAddress) {...}

auto MakeConnection(auto address, auto port) {...}

```

makes a `unique_ptr<Connection>`



```

1 auto CreateServerConnection() {
2
3     std::string cfgAddress = GetConfiguration("address");
4
5     auto address = CheckedInetAddr(cfgAddress)
6
7     std::string cfgPort = GetConfiguration("port");
8
9     auto port = htons(atoi(cfgPort.c_str()));
10
11     return MakeConnection(address, port);
12 }

```

Goals

- Maintainability:
 - easier to understand
 - cheaper to modify *correctly*
- Increase Readability
 - Increase the information content
- Identify Abstractions
 - Simplify
 - Decrease amount of code
- *Transferred Responsibility*
 - *Single Responsibility Rule: one per abstraction*
 - Put common code under common control and expertise

References

- Kate Gregory
 - "Emotional Code" ACCU Conference 2019
<https://youtu.be/uloVXmSHiSo>
 - "Naming is Hard: Let's Do Better" CppCon 2019
<https://youtu.be/MBRoCdtZOYg>
 - "Abstraction Patterns" NDC TechTown 2022
<https://youtu.be/rfIX0FzKHF0>
- Tony Van Eerd "Postmodern C++"
 - C++Now 2017: https://youtu.be/GPP64opjy_Y ("normal" talk)
 - CppCon 2017: <https://youtu.be/QTln3goa3A8> (Audio problems, but funnier and in verse a la Dr. Seuss)
- Scott Meyers
 - The Most Important Design Guideline
<https://youtu.be/sfLZ7v9gEnc>

References

- Kevlin Henney
 - "Gilding the Rose: Refactoring-Driven Development" ACCU 2023
<https://youtu.be/kTcDBYCpj7Q>
 - "Refactoring Is Not Just Clickbait" NDC London 2023
<https://youtu.be/NMPeAW2RWdc>
 - "Refactoring to Immutability" NDC London 2018
<https://youtu.be/APUCMSPiNh4>
 - "Giving code a good name" DevWeek 2015 <https://youtu.be/CzJ94TMPcD8>
- Arne Mertz: "Identifying Common Code Smells (In C++)" NDC TechTown 2022
<https://youtu.be/AriG-Wo3BYQ>
- Jason Turner: "C++ Code Smells" CppCon 2019
https://youtu.be/f_tLQl0wLUM

Act 0: The Basics

Act 1: The Problem

Act 2: The Work

Act 3: The Sell

Code Smells

Possibly more important than Fowler's list of refactorings.

- things to look for, that point towards refactoring solutions

Comments

Comments aren't a code smell in themselves, but they're used as deodorant.

```
double d1; // pre-intercooler airtemp  
double d2; // post-intercooler airtemp  
double d3; // engine torque
```

- Names should tell you *what*
- Comments should tell you *why*

Comments

Comments aren't a code smell in themselves, but they're used as deodorant.

```
double d1; // pre-intercooler airtemp  
double d2; // post-intercooler airtemp  
double d3; // engine torque
```

```
double pre_ic_temp;  
double post_ic_temp;  
double torque;
```

- Names should tell you what
- Comments should tell you *why*

Comments and Extract Function

```
Connection* CreateServerConnection() {  
    // Step 1: Get address and check that it's OK (throw if it's r  
    ...  
    // Step 2: Convert address to bytes and check that it's OK (th  
    ...  
    // Step 3: Get port and check that its OK (throw if it's not)  
    ...  
    // Step 4: Convert port to bytes  
    ...  
    // Step 5: Create connection and check that it's OK (throw if  
    ...  
    // Step 6: Return the connection  
    ...  
}
```

Comments and Extract Function

```
Connection* CreateServerConnection() {  
    // Step 1: Get address and check that it's OK (  
    ...  
    // Step 2: Convert address to bytes and check t  
    ...  
    // Step 3: Get port and check that its OK (thro  
    ...  
    // Step 4: Convert port to bytes  
    ...  
    // Step 5: Create connection and check that it'  
    ...  
    // Step 6: Return the connection  
    ...  
}
```

```
Connection* CreateServerConnection() {  
    std::string cfgAddress = Configuration("address");  
    auto address = CheckedInetAddr(cfgAddress)  
    std::string cfgPort = Configuration("port");  
    auto port = htons(atoi(cfgPort.c_str()));  
    auto connection = MakeConnection(address, port);  
    return connection;  
}
```

Comments and Extract Function

```
Connection* CreateServerConnection() {  
    // Step 1: Get address and check that it's OK (  
    ...  
    // Step 2: Convert address to bytes and check t  
    ...  
    // Step 3: Get port and check that its OK (thro  
    ...  
    // Step 4: Convert port to bytes  
    ...  
    // Step 5: Create connection and check that it'  
    ...  
    // Step 6: Return the connection  
    ...  
}
```

```
Connection* CreateServerConnection() {  
    std::string cfgAddress = Configuration("address");  
    auto address = CheckedInetAddr(cfgAddress)  
    std::string cfgPort = Configuration("port");  
    auto port = htons(atoi(cfgPort.c_str()));  
    auto connection = MakeConnection(address, port);  
    return connection;  
}
```

These are "seams" where you can cut
or "fractures" along which the code will naturally split

if blocks and Extract Function

```
1 Connection* CreateServerConnection() {
2     std::ostringstream buf;
3
4     // Step 1: Get address and check that it's OK (throw if it's not)
5     std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
6     if (cfgAddress.empty()) {
7         buf << std::format("Configuration Value Missing {}", "address");
8         Log::Instance() << buf.str();
9         throw ConnectionException(buf.str());
10    }
11
12    // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
13    auto address = inet_addr(cfgAddress.c_str());
14    if (address == -1) {
15        buf << std::format("Invalid Address: {}", cfg_address);
16        Log::Instance() << buf.str();
17        throw ConnectionException(buf.str());
18    }
19
20    // Step 3: Get port and check that its OK (throw if it's not)
21    std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
22    if (cfgPort.empty()) {
23        buf << std::format("Configuration value missing : {}", "port");
24        Log::Instance() << buf.str();
25        throw ConnectionException(buf.str());
26    }
```

if blocks and Extract Function

```
3
4 // Step 1: Get address and check that it's OK (throw if it's not)
5 std::string cfgAddress = ConfigurationManager::Instance().GetValue("address");
6 if (cfgAddress.empty()) {
7     buf << std::format("Configuration Value Missing {}", "address");
8     Log::Instance() << buf.str();
9     throw ConnectionException(buf.str());
10 }
11
12 // Step 2: Convert address to bytes and check that it's OK (throw an exception if it's
13 auto address = inet_addr(cfgAddress.c_str());
14 if (address == -1) {
15     buf << std::format("Invalid Address: {}", cfg_address);
16     Log::Instance() << buf.str();
17     throw ConnectionException(buf.str());
18 }
19
20 // Step 3: Get port and check that its OK (throw if it's not)
21 std::string cfgPort = ConfigurationManager::Instance().GetValue("port");
22 if (cfgPort.empty()) {
23     buf << std::format("Configuration value missing : {}", "port");
24     Log::Instance() << buf.str();
25     throw ConnectionException(buf.str());
26 }
27
28 // Step 4: Convert port to bytes
29 auto port = htons(atoi(cfgPort.c_str()));
```

Loops and Extract Function

Sean Parent's C++ Seasonings

- GoingNative 2013 C++ Seasoning
<https://youtu.be/W2tWOdzgXHA>
- Programming Conversations Lecture 5 part 1
<https://youtu.be/lzNtM038Jul>

A master class in "extract function". He refactored hundreds of lines of complex loops into ~5 function calls, and those functions were all one-liners that called standard algorithms

No Raw Loops

As a result, Sean Parent proposed a goal for better C++: No Raw Loops

A Raw Loop is:

- A raw loop is any loop inside a function where the function serves purpose larger than the algorithm implemented by the loop

Practically: "trivial" loops can stay.

- 2 lines of code at most
- 1 conditional
- a break if you're pushing it
- *no continue statements*

No Raw Loops

```
for (auto& x : data)  
    x *= 3;
```

```
std::ranges::transform(data, [](auto& x){x *= 3;});
```

Either is fine; too small & simple to be "raw"

No Raw Loops

```
for (auto& x : data)
    x *= 3;
```

```
std::ranges::transform(data, [](auto& x){x *= 3;});
```

Either is fine; too small & simple to be "raw"

```
auto pos = data.begin();
for ( ; i != data.end(); ++pos)
    if (*pos == my_value)
        break;
```

```
std::ranges::find(data, my_value);
```

std::ranges::find probably preferred, but I might not stop an MR for this

No Raw Loops

```
for (auto& x : data)
    x *= 3;
```

```
std::ranges::transform(data, [](auto& x){x *= 3;});
```

Either is fine; too small & simple to be "raw"

```
auto pos = data.begin();
for ( ; i != data.end(); ++pos)
    if (*pos == my_value)
        break;
```

```
std::ranges::find(data, my_value);
```

std::ranges::find probably preferred, but I might not stop an MR for this

```
auto iter = data.begin();
for ( ; iter != data.end(); ++iter) {
    if (*iter < threshold) // too small don't waste time
        continue;
    *iter *= 3;
    auto expensive = foo(*iter);
    if (expensive > min and expense < max)
        break;
}
```

NOPE.

Extact to a function called ...

"triple_nontrivial_and_find_first_acceptable_foo"?

Extract Function Seams

Seams are places where the code will cut cleanly

- Repeated or duplicated code
- Comments ("Step 1") for blocks of code
- Conditional blocks
- Raw Loops
- It Exists

Data Clump Code Smell

from Kate Gregory's Abstraction Talk

```
1 int startval, endval;  
2 int numpoints;  
3 double avg, tolerance;  
4  
5 int lat, lon, alt;  
6 bool visible;  
7 bool secure;  
8
```

Data Clump Code Smell

from Kate Gregory's Abstraction Talk

```
1 int startval, endval;  
2 int numpoints;  
3 double avg, tolerance;  
4  
5 int lat, lon, alt;  
6 bool visible;  
7 bool secure;  
8
```

← Load Bearing Blank Line

Data Clump Code Smell

from Kate Gregory's Abstraction Talk

```
1 int startval, endval;  
2 int numpoints;  
3 double avg, tolerance;  
4  
5 int lat, lon, alt;  
6 bool visible;  
7 bool secure;  
8
```

Thing 1 (statistics?)

Load Bearing Blank Line

Data Clump Code Smell

from Kate Gregory's Abstraction Talk

```
1 int startval, endval;  
2 int numpoints;  
3 double avg, tolerance;  
4  
5 int lat, lon, alt;  
6 bool visible;  
7 bool secure;  
8
```

Thing 1 (statistics?)

Load Bearing Blank Line

Thing 2 (beacon?)

Data Clump Code Smell

from Kate Gregory's Abstraction Talk

```
1 int startval, endval;  
2 int numpoints;  
3 double avg, tolerance;  
4  
5 int lat, lon, alt;  
6 bool visible;  
7 bool secure;  
8
```

Thing 1 (statistics?)

Load Bearing Blank Line

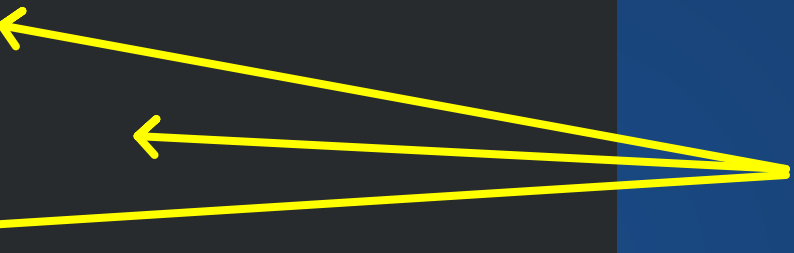
Thing 2 (beacon?)

Strong hints that there are two abstractions here; now we know what to go look for

```
1 int startval, endval;
2 int numpoints;
3 double avg, tolerance;
4
5 int lat, lon, alt;
6 bool visible;
7 bool secure;
8
```

```
1 int startval, endval;
2 int numpoints;
3 double avg, tolerance;
4
5 int lat, lon, alt;
6 bool visible;
7 bool secure;
8
```

Load-bearing lack-of-whitespace



Not All Seams Are So obvious

Sometimes it takes some analysis or trial-and-error to find the seams.

The Gilded Rose Refactoring Kata

<https://github.com/emilybache/GildedRose-Refactoring-Kata>

- 54 lines long
- 1 for loop
- 16 if
- 3 else
- No comments
- No unit tests

No easily visible seams

The Gilded Rose Refactoring Kata

Kevlin Henney

"Gilding the Rose: Refactoring-Driven Development" ACCU 2023

<https://youtu.be/kTcDBYCpj7Q>

- More structure-based seams
- Back-and-forth in direction
- Trial-and-error is inevitable
- More than one approach

```
1 void GildedRose::updateQuality() {
2     for (int i = 0; i < items.size(); i++) {
3         if (items[i].name != "Aged Brie" && items[i].name != "Backstage passes to a TAFKAL80ETC concert") {
4             if (items[i].quality > 0) {
5                 if (items[i].name != "Sulfuras, Hand of Ragnaros") {
6                     items[i].quality = items[i].quality - 1;
7                 }
8             }
9         }
10        else {
11            if (items[i].quality < 50) {
12                items[i].quality = items[i].quality + 1;
13            }
14            if (items[i].name == "Backstage passes to a TAFKAL80ETC concert") {
15                if (items[i].sellIn < 11) {
16                    if (items[i].quality < 50) {
17                        items[i].quality = items[i].quality + 1;
18                    }
19                }
20            }
21            if (items[i].sellIn < 6) {
22                if (items[i].quality < 50) {
23                    items[i].quality = items[i].quality + 1;
24                }
25            }
26        }
27    }
28 }
29
30 if (items[i].name != "Sulfuras, Hand of Ragnaros") {
31     items[i].sellIn = items[i].sellIn - 1;
32 }
33
34 if (items[i].sellIn < 0) {
35     if (items[i].name != "Aged Brie") {
36         if (items[i].name != "Backstage passes to a TAFKAL80ETC concert") {
37             if (items[i].quality > 0) {
38                 if (items[i].name != "Sulfuras, Hand of Ragnaros") {
39                     items[i].quality = items[i].quality - 1;
40                 }
41             }
42         }
43         else {
44             items[i].quality = items[i].quality - items[i].quality;
45         }
46     }
47     else {
48         if (items[i].quality < 50) {
49             items[i].quality = items[i].quality + 1;
50         }
51     }
52 }
53 }
54 }
```

Summary

Look for signs that the code is easy to cut apart "seams"

- comments
- if blocks
- No Raw Loops
- Code structure
 - Load-bearing whitespace
 - Load-bearing lack-of-whitespace
- ... and sometimes they're hidden
- ... and sometimes there aren't any

References

- Sean Parent
 - GoingNative 2013 C++ Seasoning
<https://youtu.be/W2tWOdzgXHA>
 - Programming Conversations Lecture 5 part 1
<https://youtu.be/IzNtM038Jul>
- Hyrum Wright
 - CppCon2014: "Large-Scale Refactoring @ Google"
<https://youtu.be/ZpvvmvITOrk>
 - CppCon 2018:H. Wright "Large-Scale Changes at Google: Lessons Learned From 5 Yrs of Mass Migrations"
<https://youtu.be/TrC6ROeV4GI>

Act 0: The Basics

Act 1: The Problem

Act 2: The Work

Act 3: The Sell

Goals

- Maintainability:
 - easier to understand
 - cheaper to modify *correctly*
- Increase Readability
 - Increase the information content
- Identify Abstractions
 - Simplify
 - Decrease amount of code
- *Transferred Responsibility*
 - *Single Responsibility Rule: one per abstraction*
 - Put common code under common control and expertise

Goals

- Maintainability:
 - easier to understand
 - cheaper to modify *correctly*
- Increase Readability
 - Increase the information content
- Identify Abstractions
 - Simplify
 - Decrease amount of code
- *Transferred Responsibility*
 - *Single Responsibility Rule: one per abstraction*
 - Put common code under common control and expertise

Goals

- Maintainability:
 - easier to understand
 - cheaper to modify *correctly*
 - Increase Readability
 - Increase the information content
 - Identify Abstractions
 - Simplify
 - Decrease amount of code
 - *Transferred Responsibility*
 - *Single Responsibility Rule: one per abstraction*
 - Put common code under common control and expertise
- "Macro" goals
- Fixing design problems
 - Updating design decisions
 - Mature code into production
 - Testability, robustness, cyber
 - Technical Debt to Technical Capital
 - ...

[Refactoring] is applying a series of small behavior-preserving transformations, each of which “too small to be worth doing”.

Martin Fowler

[Refactoring] is applying a series of small behavior-preserving transformations, each of which “too small to be worth doing”.

However the cumulative effect of each of these transformations is quite significant.

Martin Fowler

[Refactoring] is applying a series of small behavior-preserving transformations, each of which “too small to be worth doing”.

However the cumulative effect of each of these transformations is quite significant.

By doing them in small steps you reduce the risk of introducing errors.

Martin Fowler

[Refactoring] is applying a series of small behavior-preserving transformations, each of which “too small to be worth doing”.

However the cumulative effect of each of these transformations is quite significant.

By doing them in small steps you reduce the risk of introducing errors.

You also avoid having the system broken while you are carrying out the restructuring - which allows you to gradually refactor a system over an extended period of time.

Martin Fowler

Blast Radius

Blast Radius: how much code is affected

- How much code is affected?
 - Can you actually make that change
- How many people are affected?
 - Reviewers - can they review it?
 - Team members and team disruption
 - Users:
 - Can you even identify all the people affected?
 - It's your job to support them in making or accepting changes

Plan the work

The larger the refactor, the more coordination is needed with your team and users

- Coordinate the larger work
 - Reviewers need context for these "trivial and useless changes"

Plan the work

The larger the refactor, the more coordination is needed with your team and users

- Coordinate the larger work
 - Reviewers need context for these "trivial and useless changes"
- Keep MRs small and focused. Prefer a larger number of small MRs
 - Consider the work and cognitive load the refactor places on your team
 - "Single Responsibility" applies to MR -- keep it simple
 - Refactor your MRs

Plan the work

The larger the refactor, the more coordination is needed with your team and users

- Coordinate the larger work
 - Reviewers need context for these "trivial and useless changes"
- Keep MRs small and focused. Prefer a larger number of small MRs
 - Consider the work and cognitive load the refactor places on your team
 - "Single Responsibility" applies to MR -- keep it simple
 - Refactor your MRs
- Map the Migration Path
 - Coordinate stages of the refactoring
 - How to keep it working while the construction is in progress

Plan the Migration Path

```
auto function(old args) {  
    Logic;  
}
```

Desired Change



```
auto function(new args) {  
    Logic;  
}
```

Plan the Migration Path

```
auto function(old args) {  
    Logic;  
}
```

Overload Set

```
auto function(new args) {  
    function(equivalent old args);  
}
```

Problems can be handled as they arise

Plan the Migration Path

```
auto function(old args) {  
  Logic;  
}
```

Overload Set

```
auto function(new args) {  
  function(equivalent old args);  
}
```

New forwards to old
old is unchanged (safe!)

Problems can be handled as they arise

Plan the Migration Path

```
auto function(old args) {  
  Logic;  
}
```

Overload Set

```
auto function(new args) {  
  function(equivalent old args);  
}
```

New forwards to old
old is unchanged (safe!)

Call sites can be updated as convenient

Problems can be handled as they arise

Plan the Migration Path

"Some coconuts need to migrate"

```
auto old_fn(old args) {  
    function(equivalent new args);  
}
```



```
auto new_fn(new args) {  
    Logic;  
}
```

Plan the Migration Path

"Some coconuts need to migrate"

```
auto old_fn(old args) {  
    function(equivalent new args);  
}
```



```
auto new_fn(new args) {  
    Logic;  
}
```

Old forwards to New

Plan the Migration Path

"Some coconuts need to migrate"

```
[[deprecated]]
auto function(old args) {
  function(equivalent new args);
}
```

```
auto function(new args) {
  Logic;
}
```

Plan the Migration Path

"Some coconuts need to migrate"

```
[[deprecated]]  
auto function(old args) {  
    function(equivalent new args);  
}
```

```
auto function(new args) {  
    Logic;  
}
```

Old is deprecated
Listen for breakage
Help the latecomers

Plan the Migration Path

"Some coconuts need to migrate"

```
[[deprecated]]  
auto function(old args) {  
    function(equivalent new args);  
}
```

```
auto function(new args) {  
    Logic;  
}
```

Plan the Migration Path

"Some coconuts need to migrate"

```
auto function(new args) {  
    Logic;  
}
```

Plan the Migration Path

"Some coconuts need to migrate"

```
auto function(new args) {  
    Logic;  
}
```

Old is deleted
Listen for screams
Help the injured

Act 0: The Basics

Act 1: The Problem

Act 2: The Work

Act 3: The Sell

Part 4?

Selling to managers and customers

How to answer the question "why are we doing work that doesn't change anything"

- Invest now to save later (technical debt vs technical capital)
- It changes the probability of bad things not happening (hard to measure)

Tradeoffs in the Software Workflow - Titus Winters
- ACCU 2022

<https://youtu.be/l6Q7XaTleyI>

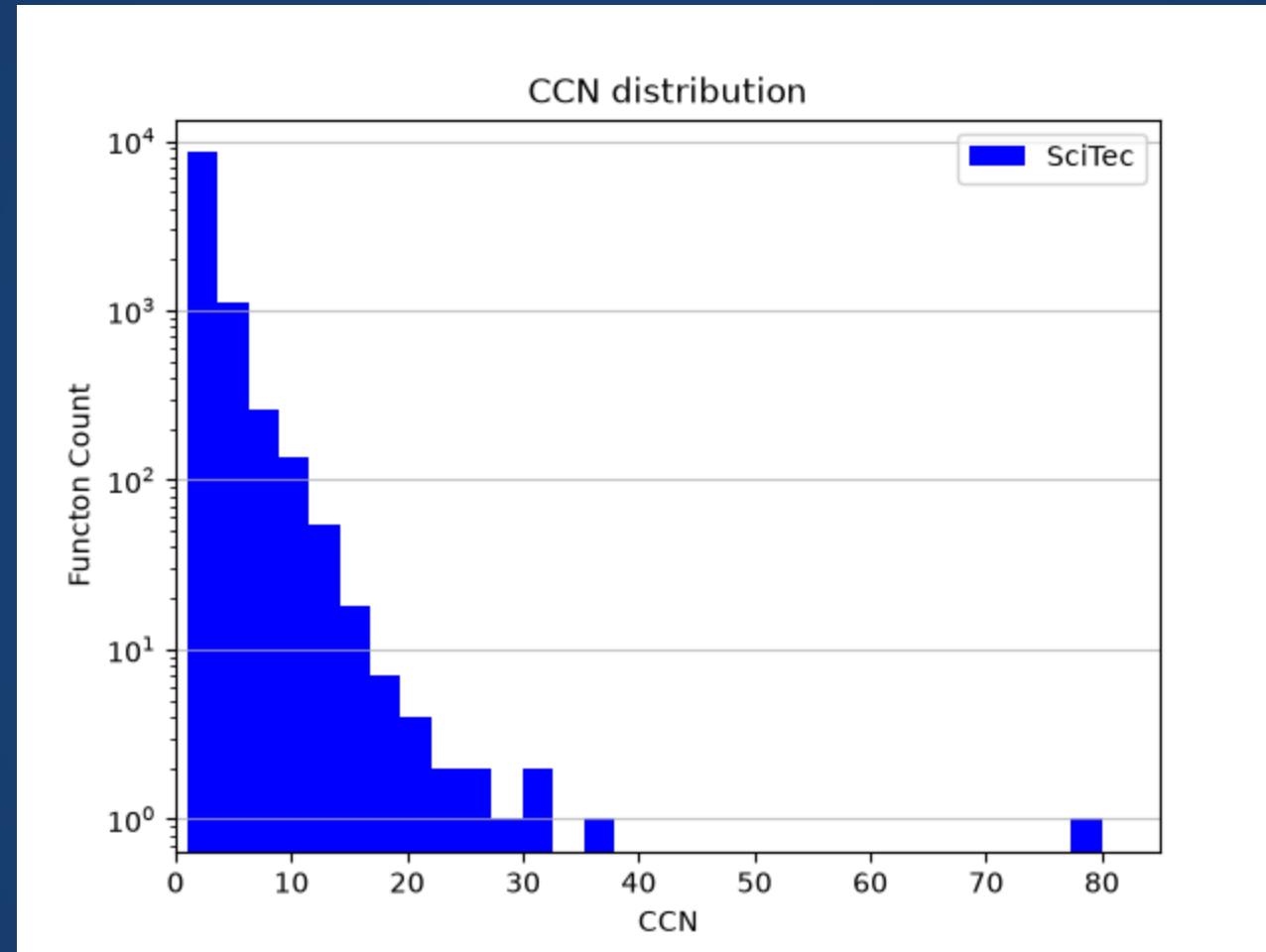
Approach 1: Reframe the question

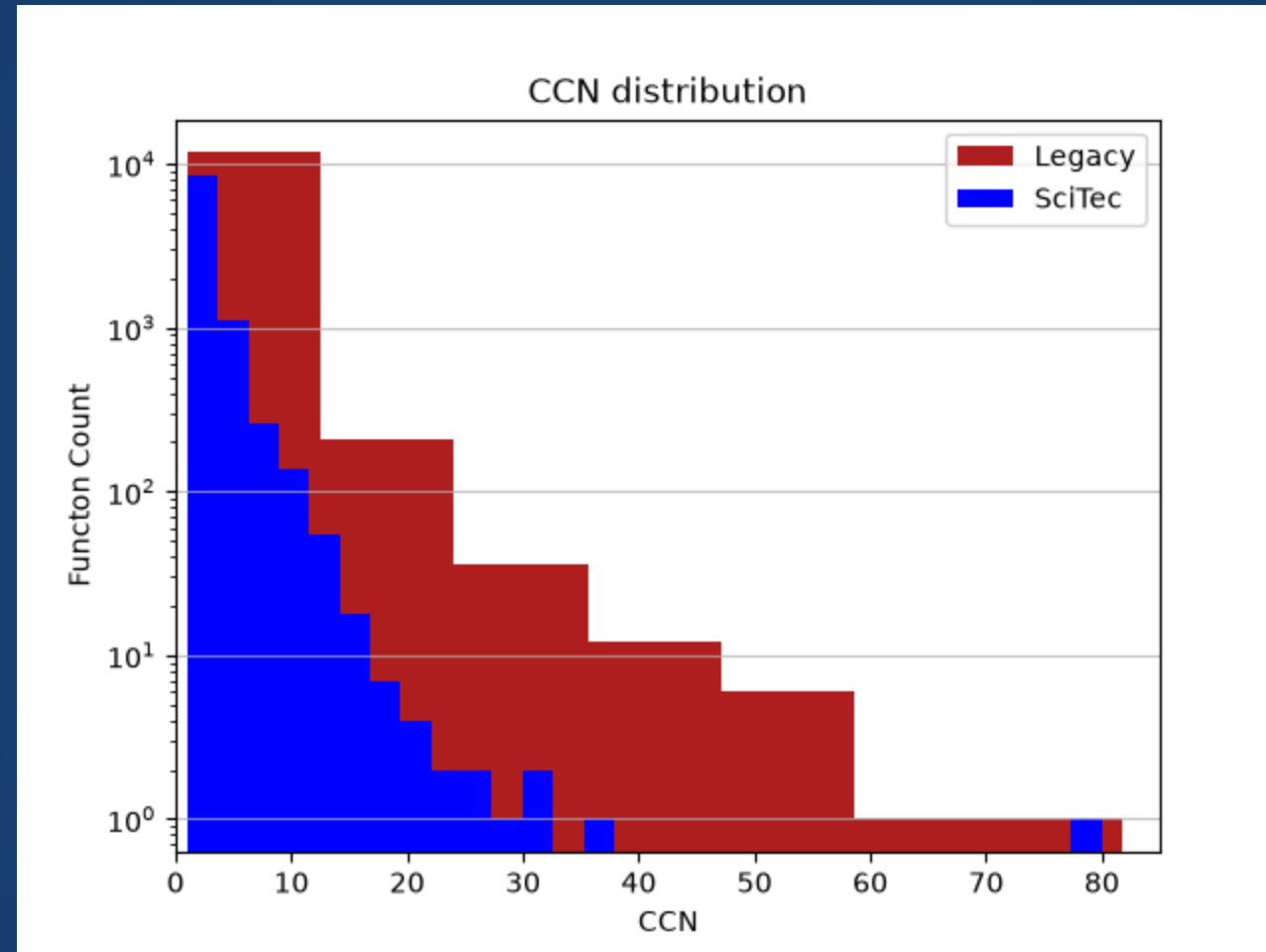
- The program doesn't go faster, but the *team* does
- The program isn't more efficient (uses less resources) but the *team* is
- The program can't do anything it couldn't do before, but the *team* can (solve new problems)

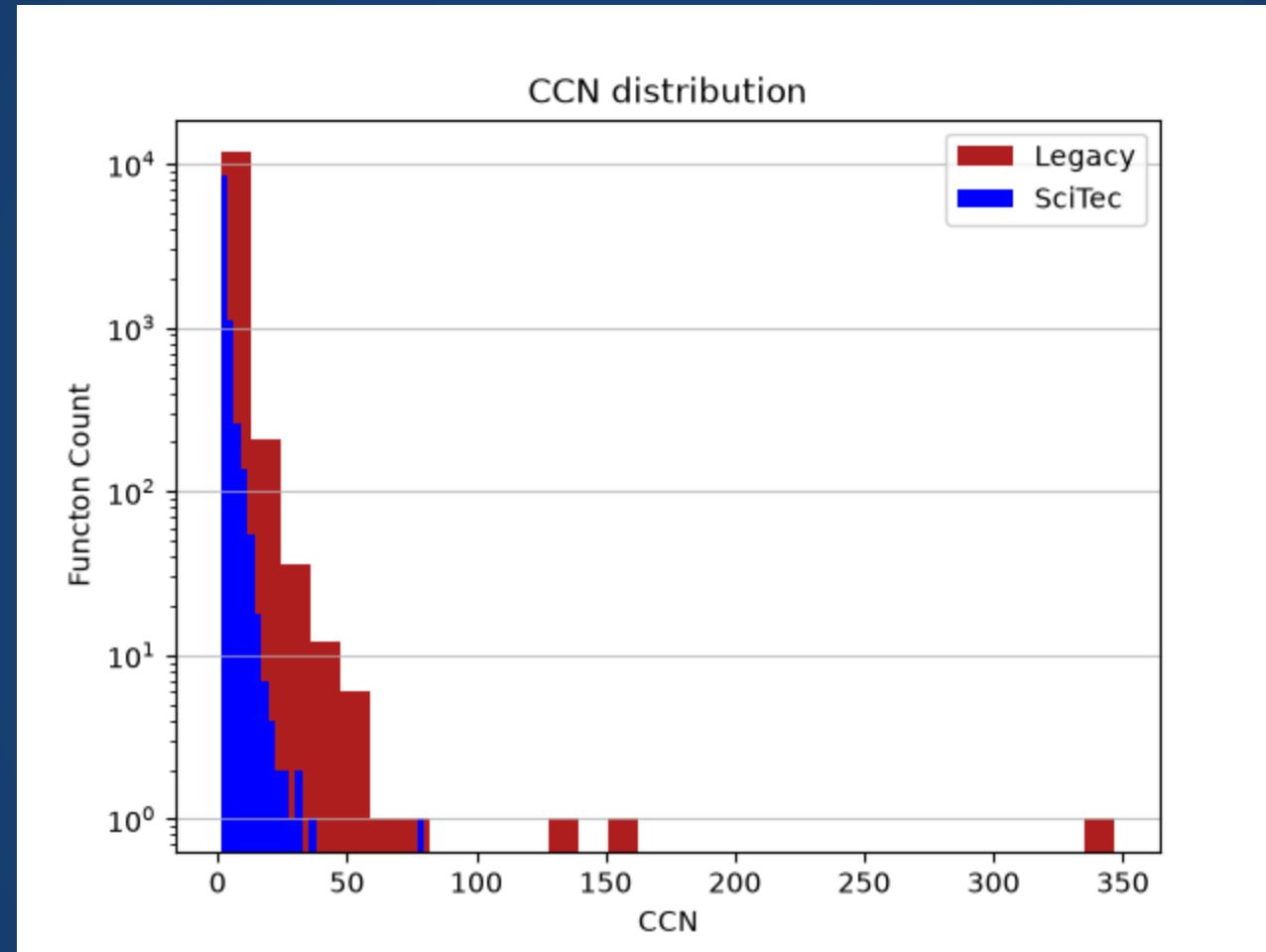
Still abstract, non-qualitative, unmeasured... how do you know it worked?

Can't measure code quality, but we can measure things that are highly correlated

- Unit test code coverage (== testability)
- In some code bases, # compiler warnings or # clang-tidy things found (decrease over time)
- CCN / Function Length is strongly correlated







Summary

Refactoring is about code quality

We need to care
-- Kate Gregory

In Debt To:

- Kate Gregory
- Kevlin Henney
- Titus Winters
- Tony Van Eerd

Thanks to :

- Lenny Maiori
- Ben Deane
- Neil Sexton (late night talk review)

References

Brian Ruth:

- "Practical Advice for Maintaining and Migrating Working Code" CppCon 2021
<https://youtu.be/CktRuMALe2A>